DBMS ASSIGNMENT SERIES

Assignment 9

Advanced SQL Queries Using Joins and Subqueries

Online Food Delivery System

SUBMITTED BY	PROGRAMME & INSTITUTION	SUBMISSION DATE
Vimal Kumar Rao	BCA (AI/ML) Alliance University	24 October 2026

A consistent relational design carried forward from Assignments 1 and 2.

IN THIS ASSIGNMENT

01	02	03	04
Join entities	Use subqueries	Build reports	Interpret results

1. Objective

Advanced queries combine related tables and use aggregate subqueries, NOT EXISTS, correlated subqueries, and derived tables to create useful reports. All queries are included in Assignment_9_Advanced_SQL_Queries.sql.

> **Output note:** Result tables were executed against an equivalent local validation database using the same sample dataset. For the submitted native MySQL evidence, execute the attached statements in MySQL Workbench; full table snapshots are also included in query_outputs.

Q1. Multi-table order overview

Joins Customer and Restaurant to Orders and optionally attaches Delivery_Partner and Payment. LEFT JOIN keeps orders whose delivery partner is not yet assigned.

```
SELECT o.Order_ID, c.Name AS Customer, r.Restaurant_Name,
    o.Order_Status, p.Payment_Method, p.Payment_Status,
    COALESCE(d.Name, 'Not assigned') AS Delivery_Partner,
    o.Order_Date
FROM Orders o
JOIN Customer c ON c.Customer_ID = o.Customer_ID
JOIN Restaurant r ON r.Restaurant_ID = o.Restaurant_ID
LEFT JOIN Delivery_Partner d ON d.Delivery_ID = o.Delivery_ID
LEFT JOIN Payment p ON p.Order_ID = o.Order_ID
ORDER BY o.Order_Date DESC, o.Order_ID DESC;
```

Order_ID	Customer	Restaurant_Name	Order_Status	Payment_Method	Payment_Status	Delivery_Partner	Order_Date
10	Priya Sharma	Wok Express	Placed	UPI	Failed	Not assigned	2026-10-24 12:30:00
9	Nisha Rao	Green Bowl Cafe	Confirmed	Wallet	Pending	Sneha Rao	2026-10-24 12:00:00
8	Rahul Nair	Dosa Junction	Delivered	Cash	Paid	Imran Khan	2026-10-24 08:45:00
7	Kabir Das	Urban Pizza Co.	Placed	UPI	Pending	Not assigned	2026-10-23 19:00:00
6	Meera Iyer	Spice Route Kitchen	Preparing	Card	Pending	Kavya Patel	2026-10-23 13:30:00
5	Arjun Reddy	Wok Express	Out for Delivery	UPI	Pending	Sneha Rao	2026-10-23 12:00:00
4	Asha Menon	Dosa Junction	Delivered	Cash	Paid	Rohan Singh	2026-10-22 20:00:00
3	Priya Sharma	Urban Pizza Co.	Delivered	UPI	Paid	Imran Khan	2026-10-22 13:20:00

Displaying 8 of 12 rows here for readability. The complete executed output is included in the accompanying query_outputs folder.

Q2. Order-line detail

Joins order headers, customers, restaurants, order lines, and menu items. Line_Subtotal is calculated as Quantity × Unit_Price.

```
SELECT o.Order_ID, c.Name AS Customer, r.Restaurant_Name,
    mi.Item_Name, oi.Quantity, oi.Unit_Price,
    (oi.Quantity * oi.Unit_Price) AS Line_Subtotal
FROM Orders o
JOIN Customer c ON c.Customer_ID = o.Customer_ID
JOIN Restaurant r ON r.Restaurant_ID = o.Restaurant_ID
JOIN Order_Item oi ON oi.Order_ID = o.Order_ID
JOIN Menu_Item mi ON mi.Item_ID = oi.Item_ID
ORDER BY o.Order_ID, mi.Item_Name;
```

Order_ID	Customer	Restaurant_Name	Item_Name	Quantity	Unit_Price	Line_Subtotal
1	Asha Menon	Spice Route Kitchen	Chicken Biryani	2	220	440
1	Asha Menon	Spice Route Kitchen	Vegetable Meals	1	150	150
2	Rahul Nair	Green Bowl Cafe	Lemon Iced Tea	2	70	140
2	Rahul Nair	Green Bowl Cafe	Veg Buddha Bowl	1	160	160
3	Priya Sharma	Urban Pizza Co.	Garlic Bread	1	119	119
3	Priya Sharma	Urban Pizza Co.	Margherita Pizza	1	249	249
4	Asha Menon	Dosa Junction	Filter Coffee	2	40	80
4	Asha Menon	Dosa Junction	Masala Dosa	2	90	180

Displaying 8 of 21 rows here for readability. The complete executed output is included in the accompanying query_outputs folder.

Q3. Top restaurants by paid line value

Uses joins, SUM, grouping, filtering, ordering, and LIMIT to rank restaurants using order lines whose payment state is Paid; cancelled orders are excluded.

```
SELECT r.Restaurant_Name,
    SUM(oi.Quantity * oi.Unit_Price) AS Paid_Line_Value
FROM Restaurant r
JOIN Orders o ON o.Restaurant_ID = r.Restaurant_ID
JOIN Order_Item oi ON oi.Order_ID = o.Order_ID
JOIN Payment p ON p.Order_ID = o.Order_ID AND p.Payment_Status = 'Paid'
WHERE o.Order_Status <> 'Cancelled'
GROUP BY r.Restaurant_ID, r.Restaurant_Name
ORDER BY Paid_Line_Value DESC, r.Restaurant_Name ASC
LIMIT 3;
```

Restaurant_Name	Paid_Line_Value
Urban Pizza Co.	866
Spice Route Kitchen	590
Dosa Junction	440

Q4. Above-average customer paid spending

Uses a derived table in a subquery to compute each paying customer's line-item spend, then returns customers whose paid spend exceeds the average among paying customers.

```
SELECT c.Customer_ID, c.Name,
      SUM(oi.Quantity * oi.Unit_Price) AS Paid_Spend
FROM Customer c
JOIN Orders o ON o.Customer_ID = c.Customer_ID
JOIN Payment p ON p.Order_ID = o.Order_ID AND p.Payment_Status = 'Paid'
JOIN Order_Item oi ON oi.Order_ID = o.Order_ID
GROUP BY c.Customer_ID, c.Name
HAVING SUM(oi.Quantity * oi.Unit_Price) > (
   SELECT AVG(customer_spending.Paid_Spend)
   FROM (
      SELECT o2.Customer_ID,
            SUM(oi2.Quantity * oi2.Unit_Price) AS Paid_Spend
      FROM Orders o2
      JOIN Payment p2 ON p2.Order_ID = o2.Order_ID AND p2.Payment_Status = 'Paid'
      JOIN Order_Item oi2 ON oi2.Order_ID = o2.Order_ID
      GROUP BY o2.Customer_ID
   ) AS customer_spending
)
ORDER BY Paid_Spend DESC;
```

Customer_ID	Name	Paid_Spend
1	Asha Menon	850

Q5. Customers with no delivered orders

Uses NOT EXISTS to return customers for whom no Delivered order can be found.

```
SELECT c.Customer_ID, c.Name
FROM Customer c
WHERE NOT EXISTS (
   SELECT 1 FROM Orders o
   WHERE o.Customer_ID = c.Customer_ID
     AND o.Order_Status = 'Delivered'
)
ORDER BY c.Customer_ID;
```

Customer_ID	Name
5	Meera Iyer
6	Kabir Das
7	Nisha Rao

Q6. Pending/failed payment follow-up

Joins payment status to order and customer details, filtering to Pending or Failed records that may require operational follow-up.

```
SELECT o.Order_ID, c.Name AS Customer, r.Restaurant_Name,
       o.Order_Status, p.Payment_Method, p.Payment_Status,
       o.Order_Date
FROM Orders o
JOIN Customer c ON c.Customer_ID = o.Customer_ID
JOIN Restaurant r ON r.Restaurant_ID = o.Restaurant_ID
JOIN Payment p ON p.Order_ID = o.Order_ID
WHERE p.Payment_Status IN ('Pending', 'Failed')
ORDER BY o.Order_Date DESC;
```

Order_ID	Customer	Restaurant_Name	Order_Status	Payment_Method	Payment_Status	Order_Date
10	Priya Sharma	Wok Express	Placed	UPI	Failed	2026-10-24 12:30:00
9	Nisha Rao	Green Bowl Cafe	Confirmed	Wallet	Pending	2026-10-24 12:00:00
7	Kabir Das	Urban Pizza Co.	Placed	UPI	Pending	2026-10-23 19:00:00
6	Meera Iyer	Spice Route Kitchen	Preparing	Card	Pending	2026-10-23 13:30:00
5	Arjun Reddy	Wok Express	Out for Delivery	UPI	Pending	2026-10-23 12:00:00

Q7. Latest order per customer

A correlated subquery compares each order date with the maximum order date for the same customer.

```
SELECT c.Customer_ID, c.Name, o.Order_ID, o.Order_Date, o.Order_Status
FROM Customer c
JOIN Orders o ON o.Customer_ID = c.Customer_ID
WHERE o.Order_Date = (
   SELECT MAX(o2.Order_Date)
   FROM Orders o2
   WHERE o2.Customer_ID = c.Customer_ID
)
ORDER BY c.Customer_ID;
```

Customer_ID	Name	Order_ID	Order_Date	Order_Status
1	Asha Menon	4	2026-10-22 20:00:00	Delivered
2	Rahul Nair	8	2026-10-24 08:45:00	Delivered
3	Priya Sharma	10	2026-10-24 12:30:00	Placed
4	Arjun Reddy	5	2026-10-23 12:00:00	Out for Delivery
5	Meera Iyer	6	2026-10-23 13:30:00	Preparing
6	Kabir Das	7	2026-10-23 19:00:00	Placed
7	Nisha Rao	9	2026-10-24 12:00:00	Confirmed

Q8. Menu size and average menu price

Uses scalar subqueries to calculate each restaurant's menu item count and average current listed price.

```
SELECT r.Restaurant_Name,
    (SELECT COUNT(*) FROM Menu_Item mi WHERE mi.Restaurant_ID = r.Restaurant_ID) AS Menu_Items,
    (SELECT ROUND(AVG(mi.Price), 2) FROM Menu_Item mi WHERE mi.Restaurant_ID = r.Restaurant_ID) AS
Average_Menu_Price
FROM Restaurant r
ORDER BY Menu_Items DESC, r.Restaurant_Name;
```

Restaurant_Name	Menu_Items	Average_Menu_Price
Dosa Junction	3	90
Green Bowl Cafe	3	120
Spice Route Kitchen	3	183.33
Urban Pizza Co.	3	232.33
Wok Express	3	173.33

2. Key Observations

- Q1 uses LEFT JOIN because delivery assignment and payment status can be optional during an order lifecycle.
- Q3 measures paid line-item value, not business profit. It excludes cancelled orders but does not include delivery fee, tax, discount, or refund accounting because those fields are outside this schema.
- Q4 returns Asha Menon as the only customer above the average paid line-item spend in the sample data. This result depends on the current fictional dataset and is not a general business conclusion.
- Q5 demonstrates a correlated anti-join pattern. A customer may still have placed or confirmed orders while having no delivered order.
- Q7 may return more than one row for a customer if multiple orders share the latest timestamp; this is intentional and documented in the SQL comment.
- SQL output is only as meaningful as the underlying constraints and business definitions. In production, transaction rules should also verify that every Order_Item belongs to the restaurant attached to its order.

3. Execution Steps

1. Run Assignment_5_Database_Creation.sql against a new database.
2. Run Assignment_6_Integrity_Constraints.sql to enforce UNIQUE, CHECK, and DEFAULT rules.
3. Run Assignment_7_Sample_Data.sql to insert parent records and child records in order.
4. Run each SELECT query in Assignment_9_Advanced_SQL_Queries.sql and compare its result grid with this report.

4. Conclusion

The advanced query set demonstrates one-to-many and many-to-many joins, optional relationships, nested aggregation, NOT EXISTS, and correlated subqueries. Together, these reports show how a relational schema supports operational monitoring and useful business summaries.